

```
In [1]: # import the code in the module and make it available to use
# "pd" or "np" act as a namespace that keeps all the attributes of the module together.
# Namespaces are useful for keeping your code readable and organized.
import pandas as pd
import numpy as np
```

Question 1

```
In [2]: # import monthly data using the pandas module
monthly = pd.read_csv('monthly2022.csv', index_col=0, header=None)
T = monthly.shape[0]
```

```
In [3]: # getting the columns
basicstats = monthly.describe().transpose() # flip over the table
basicstats = basicstats.drop(['min', 'max'],
                             axis=1) # remove the columns of "min" and "max"
```

```
In [4]: # adjust all columns to percentage except for "count"
basicstats=basicstats.apply(lambda x: x*100 if x.name!='count' else x)
basicstats
```

```
Out[4]:
```

	count	mean	std	25%	50%	75%
1	1164.0	0.919010	5.342949	-1.831312	1.286599	3.932300
2	1164.0	1.201292	7.165250	-2.046618	1.414757	4.324634
3	1164.0	1.679443	10.305532	-2.975058	0.932015	5.334085
4	1164.0	1.289393	8.812580	-2.624551	1.111770	4.615560
5	1164.0	1.134927	7.919363	-2.268328	1.080606	4.562462
6	1164.0	1.087679	7.459553	-2.459661	1.269200	4.574530
7	1164.0	1.101484	7.188280	-2.265917	1.249061	4.521342
8	1164.0	1.081399	6.974002	-2.255451	1.419656	4.600824
9	1164.0	1.072540	6.679415	-2.254117	1.418002	4.579421
10	1164.0	0.991528	6.299298	-2.198631	1.371539	4.396950
11	1164.0	1.030409	5.970618	-1.927217	1.406128	4.321670
12	1164.0	0.885144	5.121136	-1.728881	1.240414	3.847520

```
In [5]: skew = monthly.skew().rename('skew')
zstat1 = (skew/np.sqrt(6/T)).rename('zstat1')
kurtosis = monthly.kurtosis().rename('kurtosis')
zstat2 = (kurtosis/np.sqrt(24/T)).rename('zstat2')
```

```
In [6]: # combining all calculations into one dataframe
summ_stat=pd.concat([basicstats, skew, zstat1, kurtosis, zstat2], axis=1)
```

```
In [7]: # round all columns upt to 5 except for "count"
make_float = lambda x: x if x.name=='count' else round(x, 3)
summ_stat.apply(make_float)
```

```
Out[7]:
```

	count	mean	std	25%	50%	75%	skew	zstat1	kurtosis	zstat2
--	-------	------	-----	-----	-----	-----	------	--------	----------	--------

1	1164.0	0.919	5.343	-1.831	1.287	3.932	0.129	1.797	7.585	52.823
2	1164.0	1.201	7.165	-2.047	1.415	4.325	1.443	20.105	14.940	104.045
3	1164.0	1.679	10.306	-2.975	0.932	5.334	2.935	40.879	24.498	170.609
4	1164.0	1.289	8.813	-2.625	1.112	4.616	1.996	27.801	18.635	129.780
5	1164.0	1.135	7.919	-2.268	1.081	4.562	1.912	26.635	21.254	148.015
6	1164.0	1.088	7.460	-2.460	1.269	4.575	1.103	15.358	11.980	83.432
7	1164.0	1.101	7.188	-2.266	1.249	4.521	0.864	12.036	11.004	76.637
8	1164.0	1.081	6.974	-2.255	1.420	4.601	0.615	8.563	8.967	62.449
9	1164.0	1.073	6.679	-2.254	1.418	4.579	0.423	5.887	7.897	54.993
10	1164.0	0.992	6.299	-2.199	1.372	4.397	0.262	3.651	7.955	55.398
11	1164.0	1.030	5.971	-1.927	1.406	4.322	0.179	2.493	7.552	52.595
12	1164.0	0.885	5.121	-1.729	1.240	3.848	0.069	0.967	7.015	48.857

```
In [8]: # filtering out january observations
jan_only = monthly.iloc[:,12]
T = jan_only.shape[0]
```

```
In [9]: # getting the columns
basicstats = jan_only.describe().transpose()
basicstats = basicstats.drop('max', axis=1)
basicstats=basicstats.apply(lambda x: x*100 if x.name!='count' else x)
```

```
In [10]: skew = jan_only.skew().rename('skew')
zstat1 = (skew/np.sqrt(6/T)).rename('zstat1')
kurtosis = jan_only.kurtosis().rename('kurtosis')
zstat2 = (kurtosis/np.sqrt(24/T)).rename('zstat2')
```

```
In [11]: # combining all calculations into one dataframe
pd.concat([basicstats, skew, zstat1, kurtosis, zstat2], axis=1)
```

Out[11]:	count	mean	std	min	25%	50%	75%	skew	zstat1	kurtosis	zst
1	97.0	1.432382	4.748960	-7.763292	-2.620859	1.261561	4.959513	0.279057	1.122027	-0.071888	-0.144
2	97.0	5.151567	7.129972	-8.289561	0.542358	4.029265	8.897109	1.055315	4.243188	2.224109	4.471
3	97.0	12.365159	12.598187	-8.753539	4.143826	8.752395	15.790170	1.586418	6.378638	3.387983	6.811
4	97.0	8.294063	9.350790	-8.969212	2.236838	6.305435	11.666540	1.254985	5.046018	2.421718	4.868
5	97.0	6.674512	8.320049	-8.920361	1.532449	5.648835	10.050300	1.414767	5.688466	3.777919	7.595
6	97.0	5.499960	7.899300	-10.313830	0.647196	4.181040	8.767389	1.298262	5.220026	3.203720	6.440
7	97.0	4.541030	7.108869	-9.124860	-0.163285	3.928827	7.985955	1.017787	4.092297	2.174008	4.370
8	97.0	3.948563	7.122794	-9.825207	-0.770432	3.770006	8.505510	0.934246	3.756396	2.114732	4.251
9	97.0	3.103131	6.613469	-9.527435	-1.061087	2.205099	7.471714	1.005686	4.043643	2.464268	4.954
10	97.0	2.392548	5.938772	-9.022591	-1.863118	1.763824	6.441473	0.651493	2.619510	1.351890	2.717
11	97.0	2.095833	5.462412	-8.548536	-1.772083	2.148402	5.603613	0.650120	2.613987	1.364935	2.744
12	97.0	0.988703	4.576459	-7.958915	-2.421544	0.820839	4.379234	0.191563	0.770232	-0.265058	-0.532

Question 2

```
In [12]: # delete all variables and functions in namespace
%reset
# Import the modules
import pandas as pd
import numpy as np
from datetime import datetime
from tqdm.notebook import tqdm # creating progress meters or bars
import statsmodels.api as sm
```

Once deleted, variables cannot be recovered. Proceed (y/[n])? y

```
In [13]: # Define the function to calculate the fitted value from recursive OLS
def pred_from_OLS(X, Y, M):
    """Compute the next period predicted value based on recursive OLS regression.
    Args:
        X : The independant variables
        Y : The dependant variable
        M : Initial length of estimation window for out-of-sample tests
    Returns:
        Y_hat: The predicted value based on expanding windows
    """
    Y_Hat = np.full((len(Y),1), np.nan)
    Y = np.asarray(Y)
    X = np.asarray(X)
    X = sm.add_constant(X) # the first column is intercept of the regression
    # we start the index at M+1 because the first element of predicted return is at t=M+2.
    for i in range(M+1, len(Y)):
        Y1 = Y[1:i]
        X1 = X[0:i-1,:]
        reg = sm.OLS(Y1, X1, missing='drop').fit()
        # Predicted value based on the observation before
        Y_Hat[i] = reg.predict(X[i-1,:])
    return Y_Hat
```

```
In [14]: # Define the function to compute two out-of-sample measures of performance
def oos_eval(Y, Pred_Mean, Pred_Var, OOS_SSE_Hist):
    """This function computes OOS R^2 and CEV difference based on
    a vector of predicted mean and variance
    Args:
        Y: The excess returns
        Pred_Mean: The time series of predicted mean
        Pred_Var: The time series of predicted variance
        OOS_SSE_Hist: Out-of-sample sum of squares of forecast error
                     based on historical mean
    Returns:
        OOS_R2: Out-of-sample R^2
        CE: certainty equivalence of the out-of-sample return
    """
    OOS_SSE = np.sum((Y[M+1:] - Pred_Mean[M+1:])**2)
    OOS_R2 = 1 - OOS_SSE / OOS_SSE_Hist
    w0 = ((1/gam) * (Pred_Mean / Pred_Var)).clip(None, 1.5) # the maximum weight is 1.5
    r0 = Y * w0
    CE = np.mean(r0[M+1:]) - gam / 2 * np.var(r0[M+1:], ddof=1) # ddof: Degrees of Freedom
    return OOS_R2, CE
```

```
In [15]: # Create the table for in-sample R^2, out-of-sample R^2 and certainty equivlance
def GW2008(X, Y, Y_Hat, M):
    """Compute the in-sample and out-of-sample predictability
    statistics required to reproduce the plots from Goyal & Welch (2008).
    Args:
        X: The independent variables
```

```

Y : The dependant variable
Y_Hat: The vector of predicted values based on expanding windows
M : Initial length of estimation window for out-of-sample tests
Returns:
    data: a dataframe of the statistics of in-sample and out-of-sample tests
"""
# Create the benchmark using historical average

# expanding: keep the initial value fixed
Hist_Mean = np.asarray(Y.expanding().mean().shift())

# shift: shift the index axis by 1 period
Hist_Variance = np.asarray(Y.expanding().var().shift())
Y = np.asarray(Y)
X = np.asarray(X)
Y_Hat = np.asarray(Y_Hat)
X = sm.add_constant(X)

# I. In sample
Y_1 = Y[1:]
X_1 = X[0:len(Y)-1,:]
reg = sm.OLS(Y_1, X_1, missing='drop').fit()

# I.1 OLS Model (non truncated)
y1 = Y_1-np.mean(Y_1)
e1 = Y_1-reg.predict(X_1)
IS_R2 = 1-np.sum(e1**2)/np.sum(y1**2)

# I.2 OLS Model (truncated)
e1 = Y_1-reg.predict(X_1).clip(0,None)
IS_R2_T = 1-np.sum(e1**2)/np.sum(y1**2)

# II. Out of sample
# II.1 Historical mean model
OOS_SSE_Hist = np.sum((Y[M+1:]-Hist_Mean[M+1:])**2)
# Benchmark Certainty Equivalence
w0 = ((1/gam)*(Hist_Mean/Hist_Variance)).clip(None,1.5);
r0 = Y*w0
CE0 = np.mean(r0[M+1:])-gam/2*np.var(r0[M+1:],ddof=1)

# II.2 Forecast based on regression with expanding window
# call the oos_eval function
OOS_R2, CE1 = oos_eval(Y, Y_Hat, Hist_Variance, OOS_SSE_Hist)

# II.3 Truncated forecast based on regression with expanding window
OOS_R2_T, CE2 = oos_eval(Y, Y_Hat.clip(0,None), Hist_Variance, OOS_SSE_Hist)

# Return the table
data = pd.DataFrame({'$R_{IS}^2$:IS_R2','$R^2_{IS}$ (Truncated)':IS_R2_T,
                    '$R^2_{OS}$':OOS_R2,
                    '$R^2_{OS}$ (Truncated)':OOS_R2_T,'$\Delta$ CEV$:CE1-CE0,
                    '$\Delta$ CEV$ (Truncated)': CE2-CE0},index=[indep_var])*100

return data

```

```

In [16]: def Pool_Avg(Y, Y_Hat, M):
    """Compute the out-of-sample predictability statistics based on pooling average.
    Args:
        Y : The dependant variable
        Y_Hat: The vector of predicted values based on expanding windows
        M : Initial length of estimation window for out-of-sample tests
    Returns:
        data: a data frame of the statistics of the out-of-sample tests
    """
    # Create the benchmark using historical average
    Hist_Mean = np.asarray(Y.expanding().mean().shift())

```

```

Hist_Variance = np.asarray(Y.expanding().var().shift())
Y = np.asarray(Y)
Y_Hat = np.asarray(Y_Hat)

# Benchmark (Historical Average)
OOS_SSE_Hist = np.sum((Y[M+1:]-Hist_Mean[M+1:])**2)
# Benchmark Certainty Equivalence
w0 = ((1/gam)*(Hist_Mean/Hist_Variance)).clip(None,1.5);
r0 = Y*w0
CE0 = np.mean(r0[M+1:]-gam/2*np.var(r0[M+1:],ddof=1)

# Preapre for the pooling average
Y_Hat_Mean = np.mean(Y_Hat,axis=1)
Y_Hat_Median = np.full((len(Y)), np.nan)
for i in range(M+1,len(Y)):
    Y_Hat_Median[i] = np.median(Y_Hat[i,:])
# Alternative: Y_Hat_Median = np.median(Y_Hat,axis=1), this gives a warning message
OOS_Error_OLS_Pool_Mean = Y-Y_Hat_Mean
OOS_Error_OLS_Pool_Median = Y-Y_Hat_Median

# Combination forecast based on mean of various forecasts
OOS_R2_Pool_Mean, CE_Pool_Mean = oos_eval(Y, Y_Hat_Mean,
                                          Hist_Variance, OOS_SSE_Hist)

# Combination forecast based on median of various forecasts
OOS_R2_Pool_Median, CE_Pool_Median = oos_eval(Y, Y_Hat_Median,
                                              Hist_Variance, OOS_SSE_Hist)

# Return the table
data = pd.DataFrame({'$R_{OS}^2$ (Pooled Mean)':OOS_R2_Pool_Mean,
                    '$\Delta$ CEV$ (Pooled Mean)':CE_Pool_Mean-CE0,
                    '$R_{OS}^2$ (Pooled Median)':OOS_R2_Pool_Median,
                    '$\Delta$ CEV$ (Pooled Median)':CE_Pool_Median-CE0},
                    index=['Combination'])*100
return data

```

```

In [17]: # Import Data
df_monthly = pd.read_excel('PredictorData2022.xlsx',sheet_name="Monthly")
# Parse the dates properly
time = [str(d) for d in df_monthly.yyyymm]
df_monthly.index = pd.to_datetime(time,format="%Y%m")

```

D:\ProgramData\anaconda3\lib\site-packages\openpyxl\worksheet\header_footer.py:48: UserWarning: Cannot parse header or footer so it will be ignored
warn("""Cannot parse header or footer so it will be ignored""")

```

In [18]: # Variable construction
df_monthly['ExRet'] = df_monthly['CRSP_SPvw']-df_monthly['Rfree']
df_monthly['DP'] = np.log(df_monthly['D12'])-np.log(df_monthly['Index'])
df_monthly['DY'] = np.log(df_monthly['D12'])-np.log(df_monthly['Index'].shift())
df_monthly['EP'] = np.log(df_monthly['E12'])-np.log(df_monthly['Index'])
df_monthly['DE'] = np.log(df_monthly['D12'])-np.log(df_monthly['E12'])
df_monthly['tms'] = df_monthly['lty']-df_monthly['tbl']
df_monthly['dfr'] = df_monthly['corpr']-df_monthly['ltr']
df_monthly['dfy'] = df_monthly['AAA']-df_monthly['BAA']

# infl needs to be lagged one more month
df_monthly['infl'] = df_monthly['infl'].shift().copy()

# Construction of dependent and independent variables
dep_var = 'ExRet'
indep_vars = ['DE','svar','dfr','lty','ltr','infl','tms',
              'tbl','dfy','DP','DY','EP','b/m','ntis']

# Use the data from 1926/12 to 2020/12

```

```
subperiod = df_monthly.index>='1926-12-01'
df = df_monthly[subperiod]
M = 240 # Initial length of estimation window
gam = 3 # risk aversion coefficient
```

```
In [19]: # Create a prediction matrix from different predictive regressions
# Multiple predictive regression using all predictive variables excluding DE and tms
df_pred = pd.DataFrame(pred_from_OLS(df[[x for x in indep_vars
                                         if x not in ('DE', 'tms')]], df[dep_var], M))

df_pred.columns = ['All']
# Simple predictive regressions using one predictive variable at a time
for indep_var in tqdm(indep_vars):
    df_pred[indep_var] = pred_from_OLS(df[indep_var], df[dep_var], M)

0%|          | 0/14 [00:00<?, ?it/s]
```

```
In [20]: # Create the Table for the in- and out-of-sample performance
pd.options.display.float_format = '{:,.3f}'.format
# Get rid of d/e and tms to avoid perfect multicollinearity
data = GW2008(df[[x for x in indep_vars if x not in ('DE', 'tms')]],
              df[dep_var], df_pred['All'], M)
data.rename(index={data.index[0]: 'All'}, inplace=True)
for indep_var in indep_vars:
    x = GW2008(df[indep_var], df[dep_var], df_pred[indep_var], M)
    data = pd.concat([data, x])

data
```

Out[20]:

	R^2_{IS}	R^2_{IS} (Truncated)	R^2_{OS}	R^2_{OS} (Truncated)	ΔCEV	ΔCEV (Truncated)
All	2.778	2.713	-12.331	-6.524	-0.564	-0.027
DE	0.008	0.008	-0.655	-0.631	0.006	0.008
svar	0.020	0.020	-0.614	-0.604	-0.054	-0.053
dfr	0.061	0.045	-0.356	-0.278	-0.016	-0.004
lty	0.194	0.188	-0.283	0.470	-0.074	-0.015
ltr	0.224	0.210	-0.334	-0.285	0.004	0.008
infl	0.143	0.024	-0.017	0.007	-0.009	-0.008
tms	0.162	0.156	0.123	0.144	0.051	0.051
tbl	0.329	0.287	0.269	0.459	-0.009	0.001
dfy	0.256	0.256	-0.284	-0.272	-0.055	-0.055
DP	0.310	0.308	-0.272	0.064	-0.116	-0.087
DY	0.401	0.390	-0.898	-0.173	-0.148	-0.093
EP	0.317	0.440	-1.520	-0.834	-0.098	-0.025
b/m	0.645	0.644	-2.138	-1.633	-0.203	-0.162
ntis	0.436	0.317	-0.785	-0.785	0.036	0.036

```
In [21]: # Create table for combination forecasts
Pool_Avg(df[dep_var], df_pred[indep_vars], M)
```

Out[21]:

	R^2_{OS} (Pooled Mean)	ΔCEV (Pooled Mean)	R^2_{OS} (Pooled Median)	ΔCEV (Pooled Median)
Combination	0.639	0.005	0.608	0.021

